

SystemVerilog Coding Guidelines

Naming Convention

File Names

- File names should be descriptive of the hardware or purpose: HalfAdder.sv, PriorityEncoder.sv
- File names should **NOT** be named after assignment names: Lab1Problem1.sv, Project2.sv
- Add postfix _tb for testbenches with the same base name as the design being tested

Design File	design.sv
Testbench File	design_tb.sv

Module Names

- Module names should be descriptive of the hardware or purpose: HalfAdder, PriorityEncoder, Comparator, DFF
- Module names should **NOT** be named after assignment names: Lab1Problem1, Project2, Assignment3

Signal Names

- All signal names use lowercase alpha, numeric and underscore only. (Exceptions permitted for external pins and auto-generated code.)
- Use meaningful names. Avoid using arbitrary single characters (A, B, C, X, F)
- Buses are numbered high order to low order: logic [31:0] bus_signal;
- Array locations are numbered low to high. e.g. 2048x32 bit RAM: logic [31:0] mem_array [0:2047];
- Do **NOT** try to indicate the direction of the signal in the name: my_signal_in or my_signal_i
- Name all signals connecting modules in the format <src>2<dst>_<signal>. E.g. a signal carrying an address from module "processor" to module "cache" should be named "processor2cache_address"
- Include frequency for required or expected clock signals: clk50MHz, clk_100Hz

Constants

- Use localparam for defining constants
- All constant and parameter names use all capitals: BITWIDTH, WORDLENGTH, MEMSIZE

Assignment Statements

- Use non-blocking assignment statements (`<=`) in sequential elements (clocked elements)
- Use blocking assignment statements (`=`) in combinational elements (reduces to only logic gates)
- **NEVER** mix blocking and non-blocking assignments within a code block (always, initial block)

Commenting

- **ALWAYS** comment your code as you write it
- When entering multi-line comments, try to use the `/*..*/` commenting rather than putting `/*` in front of every line. It makes editing much easier.
- Use comments to describe the intent or functionality of the code, not its behavior.

Example of a useless comment:

```
i++; // Increment i
```

Example of a useful comment:

```
i++; // Move the read pointer to the next input element
```

- Preface each major section with a comment describing what it does, why it exists, how it works and assumptions on the environment.

A major section could be an always or initial block, an interface, a clocking block, a task, a function, a program, a class or a package. It should be possible to understand the function or purpose of a block of code by looking at the comments.

Indenting

- Use indenting to make code blocks easy to read and follow logically
- Be consistent with using tabs or spaces to indent. Do **NOT** mix indentations
- Indent always blocks, case statements, and if... else blocks

```
always_ff @(posedge clk) begin
    always block statements;
end
...
case(expression)
    case_1: begin
        case_1 statements;
    end
    case_2: begin
        case_2 statements;
    end
    default: begin
        default statements;
    end
endcase;
...
if (logical_test) begin
    do something;
end
else begin
    do something else;
end
```

SystemVerilog Coding

- **NO LATCHES!** Check the synthesis warnings and ensure you do not have any inferred latches.
- **NO** tri-states (never use INOUT)
- Use logic type for all behavioral descriptions (instead of wire or reg)
- Use wire type for signals used to wire modules together (port-mapping)
- Do **NOT** initialize wire type signals
- Use always_ff for sequential blocks
- Use always_comb for combination blocks
- Specify ALL cases in a case statement (use default if you don't need them all).
- Do **NOT** use casex or casez
- Do not use the bitwise operators in a Boolean context.

Example of poor use of bitwise operator:

```
reg [3:0] BYTE;  
reg VALID;  
if (BYTE & VALID) begin  
    ...  
end
```

Example of good use of Boolean operator:

```
reg [3:0] BYTE;  
reg VALID;  
if (BYTE != 4'b0000 && VALID) begin  
    ...  
end
```

- Don't instantiate modules using positional arguments. Always use the dot form:

```
my_module my_module(  
    .signal (signal),  
    .a_bus  (a_bus),  
    ...  
);
```

Best Practices for HDL Coding

Before writing any code

- List all inputs and outputs of your design
 - Create buses where useful: A 4-bit adder should have 4-bit inputs A and B rather than 8 single bit inputs for A3,A2,A1,A0 and B3, B2, B1, B0
 - Consider what speed the inputs and outputs change or need to update.
- Create a block level drawing first
 - Break the design down to blocks that do single a single function or task
 - Make a module for each unique, reusable block
 - Look at the communication paths between the blocks

While coding your design

- Comment your code as you write it.
- Do **NOT** wait until the entire device is completed to test it.
- Use simulation to test and verify each sub module as you code them.
 - Each sub module should **NOT** be a separate project in Vivado
 - Each sub module should have its own testbench for verification
 - Each sub module should be tested thoroughly before moving on
 - In testbenches, do **NOT** change input values on clock edges (rising or falling)
- Look at the elaborated design from Vivado to ensure the hardware being synthesized matches what you planned or expected.
 - Make sure no signals are left hanging or unconnected
 - Make sure no registers or bits are optimized out of the design
- Do **NOT** implement your design on hardware before testing in simulation